

首先介绍冲突操作的概念。

冲突操作是指不同的事务对同一个数据的读写操作和写写操作：

$R_i(x)$ 与 $W_j(x)$ /* 事务 T_i 读 x , T_j 写 x , 其中 $i \neq j$ */

$W_i(x)$ 与 $W_j(x)$ /* 事务 T_i 写 x , T_j 写 x , 其中 $i \neq j$ */

其他操作是不冲突操作。

不同事务的冲突操作和同一事务的两个操作是不能互换 (swap) 的。对于 $R_i(x)$ 与 $W_j(x)$, 若改变二者的次序, 则事务 T_i 看到的数据库状态就发生了改变, 自然会影响到事务 T_i 后面的行为。对于 $W_i(x)$ 与 $W_j(x)$, 改变二者的次序也会影响数据库的状态, x 的值由等于 T_j 的结果变成了等于 T_i 的结果。

一个调度 Sc 在保证冲突操作的次序不变的情况下, 通过交换两个事务不冲突操作的次序得到另一个调度 Sc' , 如果 Sc' 是串行的, 称调度 Sc 为冲突可串行化的调度。若一个调度是冲突可串行化的, 则一定是可串行化的调度。因此, 可以用这种方法来判断一个调度是否为冲突可串行化的。

[例 12.3] 今有调度 $Sc_1 = R_1(A)W_1(A)R_2(A)W_2(A)R_1(B)W_1(B)R_2(B)W_2(B)$, 假设 T_1 和 T_2 均正常提交。

可以把 $W_2(A)$ 与 $R_1(B)W_1(B)$ 交换, 得到

$R_1(A)W_1(A)R_2(A)R_1(B)W_1(B)W_2(A)R_2(B)W_2(B)$

再把 $R_2(A)$ 与 $R_1(B)W_1(B)$ 交换

$Sc_2 = R_1(A)W_1(A)R_1(B)W_1(B)R_2(A)W_2(A)R_2(B)W_2(B)$

Sc_2 等价于一个串行调度 T_1, T_2 。所以 Sc_1 为冲突可串行化的调度 (详细讲解参见二维码内容)。



微视频：冲突可
串行化判断示例

应该指出的是, 冲突可串行化调度是可串行化调度的充分条件, 不是必要条件。还有不满足冲突可串行化条件的可串行化调度。

[例 12.4] 有三个事务 $T_1 = W_1(Y)W_1(X)$, $T_2 = W_2(Y)W_2(X)$, $T_3 = W_3(X)$ 。

调度 $L_1 = W_1(Y)W_1(X)W_2(Y)W_2(X)W_3(X)$ 是一个串行调度。

调度 $L_2 = W_1(Y)W_2(Y)W_2(X)W_1(X)W_3(X)$ 不满足冲突可串行化。但是调度 L_2 是可串行化的, 因为 L_2 执行的结果与调度 L_1 相同, Y 的值都等于 T_2 的值, X 的值都等于 T_3 的值。

前面已经讲到, 商用数据库管理系统的并发控制一般采用封锁的方法来实现, 那么如何使封锁机制能够产生可串行化调度呢? 下面将要讲解的两段锁协议就可以实现可串行化调度。

12.7 两段锁协议

为了保证并发调度的正确性, 数据库管理系统的并发控制机制必须提供一定的手段来保证调度是可串行化的。目前数据库管理系统普遍采用两段封锁^①(two-phase lock, 2PL, 简称两段

① 《计算机科学技术名词》第3版中译为“两阶段锁”。

锁)协议的方法来实现并发调度的可串行性,从而保证调度的正确性。

所谓两段锁协议,是指所有事务必须分两个阶段对数据项加锁和解锁。

① 在对任何数据进行读、写操作之前,首先要申请并获得对该数据的封锁。

② 在释放一个封锁之后,事务不再申请和获得任何其他封锁。

上述两个阶段中,第一阶段是获得封锁,也称为扩展阶段。在这个阶段,事务可以申请获得任何数据项上的任何类型的锁,但是不能释放任何锁。第二阶段是释放封锁,也称为收缩阶段。在这个阶段,事务可以释放任何数据项上的任何类型的锁,但是不能再申请任何锁。

例如,事务 T_1 遵守两段锁协议,其封锁序列是

SLOCK A SLOCK B XLOCK C UNLOCK B UNLOCK A UNLOCK C;

| ←——— 扩展阶段 ———→ | | ←——— 收缩阶段 ———→ |

又如,事务 T_2 不遵守两段锁协议,其封锁序列是

SLOCK A UNLOCK A SLOCK B XLOCK C UNLOCK C
UNLOCK B;

可以证明,若并发执行的所有事务均遵守两段锁协议,则对这些事务的任何并发调度策略都是可串行化的。

例如,图 12.9 所示的调度是遵守两段锁协议的,因此一定是一个可串行化调度。可以验证如下:忽略图中的加锁操作和解锁操作,按时间的先后次序可得到如下的调度:

$L_1 = R_1(A)R_2(C)W_1(A)W_2(C)R_1(B)W_1(B)R_2(A)W_2(A)$

通过交换两个不冲突操作的次序(先把 $R_2(C)$ 与 $W_1(A)$ 交换,再把 $R_1(B)W_1(B)$ 与 $R_2(C)W_2(C)$ 交换),可得到

$L_2 = R_1(A)W_1(A)R_1(B)W_1(B)R_2(C)W_2(C)R_2(A)W_2(A)$

因此 L_1 是一个可串行化调度。

需要说明的是,事务遵守两段锁协议是可串行化调度的充分条件,而不是必要条件。也就是说,若并发事务都遵守两段锁协议,则对这些事务的任何并发调度策略都是可串行化的。但是,若并发事务的一个调度是可串行化的,不一定所有事务都符合两段锁协议。例如图 12.8(d)是可串行化的调度,但 T_1 和 T_2 不遵守两段锁协议。

另外,要注意两段锁协议和防止死锁的一次封锁法的异同之处。一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁,否则就不能继续执行,因此一次封锁法遵守两段锁协议。但是两段锁协议并不要求事务必须一次将所有

事务 T_1	事务 T_2
SLOCK A $R(A)=260$	SLOCK C $R(C)=300$
XLOCK A $W(A)=160$	XLOCK C $W(C)=250$
SLOCK B $R(B)=1000$	SLOCK A 等待
XLOCK B $W(B)=1100$	等待
UNLOCK A	等待
	$R(A)=160$
	XLOCK
UNLOCK B	$W(A)=210$
	UNLOCK C
	UNLOCK A

图 12.9 遵守两段锁协议的可串行化调度

要使用的数据全部加锁，因此遵守两段锁协议的事务可能发生死锁，如图 12.10 所示。

事务 T_1	事务 T_2
SLOCK B $R(B)=2$	
	SLOCK A $R(A)=2$
XLOCK A 等待 等待	XLOCK B 等待

图 12.10 遵守两段锁协议的事务可能发生死锁

12.8 封锁的粒度

封锁对象的大小称为**封锁粒度**(lock granularity)。封锁对象可以是逻辑单元，也可以是物理单元。以关系数据库为例，封锁对象可以是这样一些逻辑单元：属性值，属性值的集合，元组，关系，索引项，整个索引直至整个数据库；也可以是这样一些物理单元：页(数据页或索引页)，物理记录等。

封锁粒度与系统的并发度和并发控制的开销密切相关。直观地看，封锁的粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也越小；反之，封锁的粒度越小，并发度越高，但系统开销也就越大。

例如，若封锁粒度是数据页，事务 T_1 需要修改元组 L_1 ，则 T_1 必须对包含 L_1 的整个数据页 A 加锁。如果 T_1 对 A 加锁后事务 T_2 要修改 A 中的元组 L_2 ，则 T_2 被迫等待，直到 T_1 释放 A 上的锁。如果封锁粒度是元组，则 T_1 和 T_2 可以分别对 L_1 和 L_2 加锁，不需要互相等待，从而提高了系统的并行度。又如，事务 T 需要读取整个表，若封锁粒度是元组， T 必须对表中的每一个元组加锁，显然开销极大。

因此，在一个系统中如能同时支持多种封锁粒度供不同的事务选择是比较理想的，这种封锁方法称为**多粒度封锁**。选择封锁粒度时应同时考虑封锁开销和并发度两个因素，适当选择封锁粒度以求得最优的效果。一般说来，需要处理某个关系的大量元组的事务，可以以关系为封锁粒度；需要处理多个关系的大量元组的事务，可以以数据库为封锁粒度；而对于一个处理少量元组的用户事务，则以元组为封锁粒度就比较合适。

12.8.1 多粒度封锁

首先定义**多粒度树**。多粒度树的根结点是整个数据库，表示最大的数据粒度。叶结点表示